

PowerShell

IT Infrastructure Management (W04IST-SI0826G)

Wojciech Thomas, dr inż.

Department of Applied Informatics
Faculty of Information and Communication Technology

Spring 2026

This week we talk about:

- The PowerShell shell

What is PowerShell

Introduction

- a command-line shell
- a set of commands for system administration
- command naming convention Verb+Noun
- Permitted verbs: Get-Verb

- `Get-Command *Net*` - searches for all commands containing the word Net
- `Get-Help Get-NetAdapter` - shows usage information for a cmdlet
 - `-Examples` - shows only usage examples
 - `-Online` - opens the browser to the online help
- `Get-Service -Name w32time | Get-Member` - shows the properties of the object returned by a cmdlet (e.g. `Get-NetAdapter`)
- `Update-Help` - updates help files

- PowerShell
- Windows PowerShell

1 `$PSVersionTable`

Running scripts

- Default policy: `Get-ExecutionPolicy`
- Change policy: `Set-ExecutionPolicy`
- Change locally (for the current terminal):
`Set-ExecutionPolicy Bypass -Scope Process`

Tools

- Recommended editor
- Install the PowerShell extension

- Traditional editor included with Windows
- Currently receives only security updates

One-line commands

```
1 Get-Service
2 Get-Service | Where-Object Status -eq Running
3 Get-Service | Where-Object Status -eq Running | Select-Object -Property *
4 Get-Service |
5     Where-Object Status -eq Running |
6     Select-Object -Property DisplayName,StartupType
```

Formatting output

```
1 Get-Service | Format-Table
2 Get-Service | Format-List
3 Get-Service |
4     Where-Object Status -eq Running |
5     Select-Object -Property DisplayName,StartupType |
6     Format-List
```

PowerShell as a programming language

- boolean bool: \$true and \$false
- integers byte, integer, long
- floating point float, double
- decimal decimal

Simplest function

```
1 function Get-MyPSVersion {  
2     $PSVersionTable.PSVersion  
3 }  
4 Get-MyPSVersion
```

- When there are no parameters we do not use parentheses
- The last output value is the function result (you can also use return)

Using arguments in a function 1/2

- [string] - type annotation for a variable

```
1 function Test-MyConnection1 {  
2     param(  
3         [string]$ComputerName  
4     )  
5     Test-Connection $ComputerName  
6 }  
7 Get-Command Test-MyConnection1 -Syntax
```

Using arguments in a function 2/2

- [Parameter(Mandatory)] - makes the parameter required

```
1 function Test-MyConnection2 {  
2     param(  
3         [Parameter(Mandatory)]  
4         [string]$ComputerName  
5     )  
  
6     Test-Connection $ComputerName  
7 }
```

If statement

- the else section is optional

```
1 $condition = $false  
2 if($condition){  
3     Write-Output "True..."  
4 }else{  
5     Write-Output "False..."  
6 }
```

Using arguments in a function 3/3

- [ValidateNotNullOrEmpty()] - parameter cannot be empty
- if the user does not provide a value, the parameter will take a default value

```
1 function Test-My3Connection {  
2     param(  
3         [ValidateNotNullOrEmpty()]  
4         [string]$ComputerName = $env:COMPUTERNAME  
5     )  
  
6     Test-Connection $ComputerName
```

Comparison operators 1/2

- -eq and -ieq equality (case-insensitive)
- -ceq equality (case-sensitive)
- -ne inequality (also -ine and -cne)
- -gt → >
- -ge → ≥
- -lt → <
- -le → ≤

```
1 if($env:COMPUTERNAME -eq DC1 ){  
2     Write-Output "I'm running on DC1"  
3 }
```



- -like wildcard comparison using ? (single character) and * (any sequence)
if(\$name -like *soft){...}
- -match regular expression comparison if (\$dnsname -match '.*\\.pl'){...}
- -is checks whether a variable is of a given type if(\$dnsname -is [string]){...}

Classic for loop

```

1 # Create user accounts
2 for( $i = 1; $i -lt 5; $i++ ){
3     New-ADUser "User$i"
4 }

```

Foreach loop

Equivalent to for but iterates a collection

```

1 # Start five virtual machines
2 $ids = 1..5
3 foreach ($id in $ids){
4     Start-VM "srv$id"
5 }
6
6 # Check TCP/IP connectivity to hosts
7 $srv = 'sr1', 'sr2', 'dc1'
8 foreach ($s in $srv){
9     Test-Connection $s
10 }

```

Foreach-Object loop

Commonly used in the pipeline

```

1 Get-NetAdapter | Foreach-Object {$_.name}

```

```
1 $n=1
2 While($n -le 10){
3     Write-Output $n
4     $n++
5 }

6 $n=10
7 Do {
```

■ See you next week

Bibliography

- [1] J. Petty, T. Plunk, T. Leonhardt, D. Jones, and J. Hicks, *Learn PowerShell in a month of lunches*. Manning, 2022.

